

FAKE JOB & INTERNSHIP DETECTION SYSTEM

BYOP – Programming in Java

Submitted By : Sahil Singh
Registration Number : 24BAI10461
Slot : E11 + E12
Date of Submission : 29/03/26

**Department of Computer Science and Engineering
(AIML)
VIT Bhopal University**

1. Introduction

Nowadays students are really dependent on internet for finding jobs and internships. Sites like LinkedIn, Internshala and many other job portals have made it very convenient to search for opportunities sitting at home. But along with the genuine posts, there are also many fake ones which target freshers and students who don't have much experience.

These fake postings mostly offer very high salary which is not realistic, or they ask students to pay some registration fee or processing charges before joining. Because of such things many students end up losing money and also feel very demotivated after getting cheated like this.

So I thought of making something which can actually help in this situation. The idea of this project came from this very problem. I wanted to build a system using Java which can take the details of a job posting and then tell the user whether it looks safe or suspicious.

The system basically checks some specific things in the job like the company name, the email id provided, salary offered and whether any payment is being asked or not. Based on all these checks it gives a result to the user so they can decide whether to proceed with that job or not.

The whole project is built using Java because that is what we have been studying in this course. It uses concepts like OOP, ArrayList and file handling which are all important topics we have covered. I tried to keep the system simple enough that any student can use it without any technical knowledge.

2. Problem Statement

The number of online job platforms has gone up a lot in recent years and with that the fake job postings have also increased. Students who are just starting out in their career are the most common targets because they don't know much about how the hiring process works.

The main issue is that right now there is no easy or quick way for a student to verify if a job posting is real or not. Scammers are quite smart, they make their fake posts look very professional and similar to real job offers. So it becomes really hard to identify the difference.

There are several problems that students face because of this situation:

- Most students don't know what the common signs of a scam job look like
- It is difficult to verify whether a company actually exists or not
- Job descriptions are often written in a confusing way to mislead applicants
- Some fake jobs ask for money which leads to direct financial loss
- Students waste a lot of their time applying to fake jobs and waiting for responses that never come

Because of all these reasons a system is needed that can look at job related information and give a quick assessment of how risky that job posting is. The system should be fast and should not require the user to have any special knowledge to operate it.

3. Objective

The main objective of making this project is to develop an application in Java which can help students to detect whether a job or internship posting is fake or not. The system does this by looking at the details of the job and checking for any suspicious signs.

The specific objectives that this project aims are listed below:

- To take input from the user about job details like company name, email address, job description, salary and whether any payment is required
- To check all these details against certain conditions that are defined in the code
- To calculate a risk score depending on how many suspicious conditions are matched
- To show the result to the user by classifying the job as Safe, Suspicious or Highly Risky
- To help students take better and more informed decision before applying to any job

Apart from the main functionality, this project also shows how programming concepts that we learnt in class can be applied to solve a real problem. Things like OOP, data structures and file handling are all used in this project in a practical way.

In short the goal was to make something useful and simple, something that a student can actually open and use in their day to day life whenever they come across a job posting that looks a bit off.

4. Methodology

For building this system I decided to go with a rule based approach. I did not use any machine learning or AI because that would have been too complex for this level of project. Instead I defined some simple rules or conditions and the system checks the job details against those rules.

The working of the system can be explained in the following steps:

- First the user enters all the details about the job posting like the company name, the email id, the job description, the salary being offered and whether money is being asked or not.
- After taking the input the system then checks each field against certain conditions which are coded in the JobAnalyzer class.

The conditions that the system checks are things like:

- Whether the company name field is empty or not
- Whether the email address looks valid or not
- Whether the job is asking for any payment from the applicant
- Whether the salary mentioned is unrealistically very high
- Whether the job description is too short or does not make proper sense

Each condition that matches adds some points to a risk score. At the end when all the conditions have been checked the total score is used to decide the final result. If the score is low the job is classified as Safe, if it is medium then Suspicious and if it is very high then Highly Risky.

This whole approach is quite straightforward and does not need any heavy computation. It gives results almost instantly which is exactly what a student would need.

5. Implementation

The project is coded in Java and the structure follows OOP principles. I have divided the whole code into three separate classes and each class handles a specific part of the functionality. This makes the code cleaner and also easier to understand.

Job Class

The Job class is basically used to hold all the information about a single job posting. It has fields like the company name, email, description, salary and a boolean variable which tells whether the job requires any payment or not. This class acts like a container or a model for storing job data in an organised way.

JobAnalyzer Class

This is the most important class in the whole project. It takes the Job object as input and then runs all the checks on it. For every condition that matches a certain number of points are added to the risk score. After all checks are done the class looks at the final score and returns whether the job is Safe, Suspicious or Highly Risky.

Main Class

The Main class is where the program actually starts running. It shows a simple menu to the user with three options:

- Add a new job posting to check
- Analyse all the jobs that have been added
- Exit the program

I used ArrayList here to store multiple job entries so that the user can add more than one job and analyse all of them together. There is also file handling in this class which writes all the job data to a file called jobs.txt so the data does not get lost when the program is closed.

Using three separate classes made the code very modular and I feel it is much easier to read and debug compared to writing everything in one single class.

6. Features

The system has some really useful features which I think make it practical and easy to use for students. I will explain each of them below.

The first thing is the menu driven interface. When you run the program it shows you a menu with options. You don't have to remember any commands or anything like that. You just pick a number and the program does the rest. This makes it very beginner friendly.

Then there is the ability to store multiple jobs. Using ArrayList the program can hold as many job entries as you want in a single session. So if you have five or ten jobs to check you can add all of them first and then analyse all at once instead of running the program again and again.

The risk classification is the main feature of the whole system. After you add a job and run the analysis the program will tell you clearly whether that job is Safe, Suspicious or Highly Risky. This gives students a clear idea of whether they should apply or not.

File handling is another good feature. All the job data that the user enters gets saved to a text file called jobs.txt automatically. This means even if you close the program the records are still there in the file. It is a simple form of data persistence.

Lastly the analysis is very fast. There are no complex calculations involved. The system just checks a few simple conditions and gives the result instantly. No waiting time at all.

7. Challenges Faced

While building this project I ran into quite a few problems. Some of them were small and got fixed quickly but some took me a while to figure out. I will mention the main ones here.

The biggest challenge for me was deciding what conditions to use for detecting fake jobs. I had to think carefully about what actually makes a job posting look suspicious. I did some research, looked at some real scam examples online and then decided on the conditions to use. It was not as easy as it sounds because if the conditions are too strict then even real jobs might get flagged.

Another problem I faced was with taking input from the user using Scanner. Whenever I had to take a mix of string and integer inputs one after the other the program would sometimes skip an input or throw an error. It took me some time to fix this by properly handling the newline character after integer inputs.

Making the menu loop work properly was also a bit tricky. I wanted the menu to keep showing until the user chooses to exit but in the beginning there were some issues with the loop not breaking at the right time. I had to adjust the conditions inside the loop a few times before it worked correctly.

File handling gave me some problems too. In the beginning whenever I ran the program again the old data in jobs.txt was getting overwritten. I had to look into how to open the file in append mode so that new data gets added to the existing content rather than replacing it.

Overall though solving these problems actually helped me learn more. Every bug I fixed taught me something new about Java.

8. Learning Outcomes

This project taught me a lot of things which I think will be very useful going forward. Working on a complete application from scratch is a very different experience compared to just solving small coding problems.

First of all I got a much better understanding of OOP concepts. I had studied about classes and objects in theory but using them to actually structure a project made things a lot clearer. I now understand why we divide code into separate classes and how they interact with each other.

I also learnt how to use ArrayList properly. Before this project I had only used simple arrays but ArrayList is much more flexible especially when you I know in advance how many items you will need to store. Using it to manage multiple job entries was a good practical exercise.

File handling was another thing I learnt properly through this project. Reading and writing to files is a very common requirement in real applications and now I have a decent idea of how to do it in Java. The concept of data persistence, meaning the data survives even after the program closes, was something I understood properly only after doing this project.

My problem solving skills also improved a lot. There were several times when the code was not working and I had to sit and think about what exactly was going wrong. Debugging step by step is something I got better at.

And overall the project gave me a feel of what it is like to build a complete software application. Not just writing a function or solving one problem but thinking about the full structure, the user experience and how everything connects together.

9. Future Scope

The current version of the system works well for basic detection but there is definitely a lot of room for improvement. I have some ideas about how this project can be made better in the future.

The biggest upgrade would be to add Machine Learning to the system. Right now the detection is based on fixed rules that I manually defined. But with ML the system can be trained on thousands of real and fake job postings and it will learn to identify patterns on its own. This would make the detection much more accurate especially for tricky cases.

Another thing I want to add in future is a proper GUI. Right now the program runs on the command line which is fine for someone who is comfortable with that but most students prefer a visual interface. A GUI with proper input boxes and buttons would make the system much more user friendly.

The current system saves data to a plain text file which works but is not very efficient. If the system is to be used on a larger scale it would be better to connect it to a proper database like MySQL. That would make it much faster to search and retrieve records.

There is also an idea to connect this system with actual job portals using their APIs. If that is possible then the system could directly fetch job listings from sites and automatically check them for red flags instead of requiring the user to manually enter all the details.

Some other features that could be added later include user login, tracking history of jobs checked, and also maybe a feature to report fake jobs so that other users can also be warned.

10. Conclusion

To wrap things up, this project was a really good experience for me. I got to work on something that actually solves a real problem that students like me face on a daily basis. Fake jobs are a serious issue and even a basic tool like this can help someone avoid getting scammed.

The system uses a rule based approach to check job postings and classify them as Safe, Suspicious or Highly Risky. It looks at things like the company name, email format, salary and whether money is being demanded. Based on these checks it gives a risk score and shows the result to the user.

I know the system is not perfect. Since it uses fixed rules it might sometimes give wrong results for edge cases. But as an initial screening tool it does its job well. It gives students a starting point to decide whether a job is worth pursuing or not.

All the Java concepts I learnt during the course like OOP, ArrayList and file handling were used in this project and that made the learning more meaningful. I feel much more confident about my Java skills after completing this.

With more time and resources this project can definitely be improved and turned into something more powerful. But even in its current form I think it is a useful and practical tool for students.